```
import pandas as pd
import datetime
import streamlit as st
import urllib.parse
import webbrowser
import time
import numpy as np  # Added for realistic distribution

# 1. System Constants
TOTAL_PATIENTS = 400
DAYS_CYCLE = 15
UNITS_PER_PATIENT = 2
TARGET_POOL = 9600

# 2. Fully Corrected Data Generator Engine
@st.cache_data
def generate_system_data():
    base_date = datetime.date(2026, 1, 1)

    # 400 Patient Database
    patient_data = {
        "Patient ID": [f"PT-{i:03d}" for i in range(1, TOTAL_PATIENTS + 1)],
        "Patient Name": [f"Patient Name #{i:03d}" for i in range(1, TOTAL_PATIENTS + 1)],
        "Blood Group": ["O+", "A+", "B+", "AB+", "O-", "A-", "B-", "AB-"] * 50,
        "Baseline Slot": [i % DAYS_CYCLE for i in range(TOTAL_PATIENTS)]
    }
    df_patients = pd.DataFrame(patient_data)
```

```
    # FIX: Generating varied history so ALL blood groups have eligible donors (>90 days rest)
    np.random.seed(42)  # For consistent mock data generation
    random_days_agone = np.random.randint(10, 150, size=TARGET_POOL)

    donor_data = {
        "Donor ID": [f"DN-{i:04d}" for i in range(1, TARGET_POOL + 1)],
        "Donor Name": [f"Donor Card #{i:04d}" for i in range(1, TARGET_POOL + 1)],
        "Contact Number": [f"9198480{i:05d}" for i in range(1, TARGET_POOL + 1)],
        "Blood Group": ["O+", "A+", "B+", "AB+", "O-", "A-", "B-", "AB-"] * 1200,
        "Last Donation Date": [base_date - datetime.timedelta(days=int(random_days_agone[i])) for i in range(TARGET_POOL)]
    }
    df_donors = pd.DataFrame(donor_data)

    inv_data = {
        "BloodGroup": ["O+", "A+", "B+", "AB+", "O-", "A-", "B-", "AB-"],
        "Units_In_Stock": [45, 12, 50, 8, 30, 15, 60, 25],
        "Min_Required": [25, 25, 25, 25, 25, 25, 25, 25]
    }
    df_inv = pd.DataFrame(inv_data)

    return df_patients, df_donors, df_inv

df_patients, df_donors, df_inv = generate_system_data()

# 3. Streamlit Interface Initialization & Premium GUI Styling
st.set_page_config(
    page_title="Thalassemia Supply Planner",
```

```
    page_icon="🩸",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Premium Clinical Healthcare Theme Injector (Custom CSS)
st.markdown("""
    <style>
    /* Premium Clinical Background Gradient */
    .stApp {
        background: radial-gradient(circle at 10% 20%, rgba(242, 246, 250, 1) 0%, rgba(225, 235, 245, 0.7) 90.1%) !important;
        font-family: 'Inter', -apple-system, sans-serif !important;
    }

    /* Clean Sidebar Styling Override */
    [data-testid="stSidebar"] {
        background-color: rgba(255, 255, 255, 0.65) !important;
        backdrop-filter: blur(10px) !important;
        border-right: 1px solid rgba(225, 235, 245, 0.8);
    }

    /* Tab Styling Overrides */
    .stTabs [data-baseweb="tab-list"] {
        gap: 16px;
        padding: 4px;
        background-color: rgba(255, 255, 255, 0.4);
        border-radius: 8px;
```

```
}
.stTabs [data-baseweb="tab"] {
    padding: 10px 24px;
    background-color: transparent;
    border-radius: 6px;
    border: none !important;
    font-weight: 600;
    color: #495057;
    transition: all 0.2s ease;
}
.stTabs [data-baseweb="tab"][aria-selected="true"] {
    background-color: #dc3545 !important;
    color: white !important;
    box-shadow: 0 4px 12px rgba(220, 53, 69, 0.25);
}

/* Elegant Frosted Glass Base Metric Wrap Cells */
.stable-stock, .critical-stock {
    margin-bottom: 12px;
    display: block;
    border-radius: 12px;
    padding: 16px;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.stable-stock:hover, .critical-stock:hover {
    transform: translateY(-2px);
}
```

```
/* Distinct Low Stock Dynamic Overrides */
.critical-stock {
   background: rgba(255, 235, 238, 0.7) !important;
   border: 1px solid rgba(239, 154, 154, 0.5) !important;
   border-left: 6px solid #c62828 !important;
   box-shadow: 0 4px 15px rgba(198, 40, 40, 0.05);
}

/* Distinct Stable Level Styling Overrides */
.stable-stock {
   background: rgba(232, 245, 233, 0.7) !important;
   border: 1px solid rgba(165, 214, 167, 0.5) !important;
   border-left: 6px solid #2e7d32 !important;
   box-shadow: 0 4px 15px rgba(46, 125, 50, 0.05);
}

/* Custom CSS HTML Pure Progress Trackers */
.html-progress-bg {
   background-color: rgba(0, 0, 0, 0.06);
   border-radius: 10px;
   width: 100%;
   height: 8px;
   margin: 8px 0;
   overflow: hidden;
}
.critical-stock .html-progress-fill {
   background-color: #c62828 !important;
   height: 100%;
```

```
    border-radius: 10px;
  }
  .stable-stock .html-progress-fill {
    background-color: #2e7d32 !important;
    height: 100%;
    border-radius: 10px;
  }

  .stock-label {
    font-weight: 700;
    font-size: 15px;
    color: #2c3e50;
    margin-bottom: 2px;
  }
  .stock-value {
    font-size: 26px;
    font-weight: 800;
    color: #1a1a1a;
    margin-bottom: 2px;
  }
  .stock-deficit { color: #c62828; font-weight: 700; font-size: 13px; margin-top: 4px;}
  .stock-surplus { color: #2e7d32; font-weight: 700; font-size: 13px; margin-top: 4px;}
  </style>
""", unsafe_allow_html=True)

# ==================== PERFECTLY CENTERED HEADER SECTION ====================
left_pad, header_center, right_pad = st.columns([1, 10, 1])
```

```
with header_center:
   st.markdown("<h1 style='text-align: center; font-size: 58px; font-weight: 800; color: #1a1a1a; margin: 25px 0 10px 0; padding: 0; letter-spacing: -0.5px;'> 🩸 Thalassemia Care Dashboard</h1>", unsafe_allow_html=True)


   brand_left, logo_box, text_box, brand_right = st.columns([2.4, 0.6, 6.5, 2.5])
   with logo_box:
      try:
         st.image("PGI_logo.jpg", width=80)
      except Exception:
         st.markdown("<h1 style='margin: 0; text-align: center;'> 🏥 </h1>", unsafe_allow_html=True)
   with text_box:
      st.markdown("<h2 style='margin: 12px 0 0 0; padding: 0 0 0 10px; font-size: 25px; font-weight: 600; color: #2c3e50; line-height: 1.25;'>Department of Transfusion Medicine, PGIMER, Chandigarh</h2>", unsafe_allow_html=True)


st.divider()
#
================================================================================


# --- DATE CONTROL INTERACTION PANEL ---
st.sidebar.header(" 🏥 Operational Control Desk")
selected_date  = st.sidebar.date_input(
   label="Choose Target Planning Date",
   value=datetime.date(2026, 1, 28),
   min_value=datetime.date(2026, 1, 1),
   max_value=datetime.date(2026, 12, 31)
```

```
)

# 4. Infinite Rolling Cycle Engine
base_datetime = pd.to_datetime(datetime.date(2026, 1, 1))
selected_datetime = pd.to_datetime(selected_date)
current_day_name = selected_datetime.day_name()
st.sidebar.markdown(f"**Selected Day Status:** `{current_day_name}`")

days_since_start = (selected_datetime - base_datetime).days
current_slot_today = days_since_start % DAYS_CYCLE

if current_day_name == "Sunday":
    patients_today = df_patients[df_patients['Patient ID'] == 'NONE']
else:
    patients_today = df_patients[df_patients['Baseline Slot'] == current_slot_today].copy()

days_until_sunday = (6 - selected_datetime.weekday()) % 7
sunday_datetime = selected_datetime + pd.Timedelta(days=days_until_sunday)
sunday_slot = (days_since_start + days_until_sunday) % DAYS_CYCLE

if current_day_name == "Friday":
    sunday_patients = df_patients[df_patients['Baseline Slot'] == sunday_slot]
    offloaded_patients = sunday_patients.iloc[::2]
    patients_today = pd.concat([patients_today, offloaded_patients])
elif current_day_name == "Saturday":
    sunday_patients = df_patients[df_patients['Baseline Slot'] == sunday_slot]
    offloaded_patients = sunday_patients.iloc[1::2]
    patients_today = pd.concat([patients_today, offloaded_patients])
```

```
units_needed_today = len(patients_today) * UNITS_PER_PATIENT

donors_needed_today = units_needed_today


# Process donor eligibility statuses relative to selected date

donor_last_dates = pd.to_datetime(df_donors['Last Donation Date'])

df_donors['Days Since Donation'] = (selected_datetime - donor_last_dates).dt.days

df_donors['Status'] = df_donors['Days Since Donation'].apply(lambda x: 'Eligible' if x >= 90 else 'Ineligible')

eligible_donors_today = df_donors[df_donors['Status'] == 'Eligible']


tab1, tab2 = st.tabs([" 📊 Live Allocation Dashboard", " 🗄️ Master Institutional Registries"])


with tab1:

    kpi1, kpi2, kpi3 = st.columns(3)

    kpi1.metric(label="Patients Scheduled Today", value=f"{len(patients_today)} Patients")

    kpi2.metric(label="Demand Window Targets", value=f"{units_needed_today} Units Required")


    surplus_value = len(eligible_donors_today) - donors_needed_today

    kpi3.metric(

        label="Eligible Pool Available",

        value=f"{len(eligible_donors_today)} Donors",

        delta=f"{surplus_value} Available Surplus" if surplus_value >= 0 else f"{surplus_value} Resource Deficit",

        delta_color="normal" if surplus_value >= 0 else "inverse"

    )
```

```
    st.divider()


    # Live Inventory Sub-Grid Row
    st.markdown("### 📦 Department Storage Units Matrix")
    inv_cols = st.columns(8)
    for idx, inv_row in df_inv.iterrows():
        stock_delta = int(inv_row['Units_In_Stock'] - inv_row['Min_Required'])


        max_capacity_scale = 75.0
        fill_percentage = min((float(inv_row['Units_In_Stock']) / max_capacity_scale) * 100,
100.0)


        with inv_cols[idx % 8]:
            card_style = "critical-stock" if stock_delta < 0 else "stable-stock"


            html_card = f"""
            <div class="{card_style}">
                <div class="stock-label">Group {inv_row['BloodGroup']}</div>
                <div class="stock-value">{inv_row['Units_In_Stock']} Units</div>
                <div class="html-progress-bg">
                    <div class="html-progress-fill" style="width: {fill_percentage}%;"></div>
                </div>
            """
            if stock_delta < 0:
                html_card += f'<div class="stock-deficit">↓ {stock_delta} Units vs
Min</div></div>'
            else:
```

```
            html_card += f'<div class="stock-surplus">↑ +{stock_delta} Units vs Min</div></div>'

        st.markdown(html_card, unsafe_allow_html=True)

    st.divider()

    col1, col2 = st.columns(2, gap="large")
    with col1:
        st.subheader(f"📋 Appointment Docket ({current_day_name})")
        if current_day_name == "Sunday":
            st.success("🎉 Center Closed: Zero patients enrolled on Sundays.")
        elif not patients_today.empty:
            if current_day_name in ["Friday", "Saturday"]:
                st.warning(f"⚖️ Balanced Load Alert: Includes Sunday allocations.")
            st.dataframe(patients_today[['Patient ID', 'Patient Name', 'Blood Group']], use_container_width=True, hide_index=True)
        else:
            st.info("No patients scheduled.")

    with col2:
        st.subheader("🎯 Priority Donor Outreach Matrix")
        if current_day_name == "Sunday":
            st.info("Operations paused.")
            display_df = pd.DataFrame()
        else:
            call_sheet = eligible_donors_today.sort_values(by='Days Since Donation', ascending=False)
            if not call_sheet.empty and not patients_today.empty:
```

```
        # Extracts required groups from today's appointment log frame
        required_groups = patients_today['Blood Group'].unique()
        matched_call_sheet = call_sheet[call_sheet['Blood
Group'].isin(required_groups)].copy()


        action_rows = []
        for _, row in matched_call_sheet.iterrows():
          msg = (f"Hello {row['Donor Name']}. Your blood group ({row['Blood Group']}) "
              f"is urgently required today at the Thalassemia Care Center.")
          encoded_msg = urllib.parse.quote(msg)
          whatsapp_url = f"https://wa.me{row['Contact
Number']}?text={encoded_msg}"
          action_rows.append({
            "ID": row['Donor ID'], "Name": row['Donor Name'], "Blood Group": row['Blood
Group'],
            "Phone": row['Contact Number'], "Days Rested": row['Days Since Donation'],
"WhatsApp Link": whatsapp_url
          })
        display_df = pd.DataFrame(action_rows)


        if not display_df.empty:
          st.write("### Targeted Action Sheet")
          st.data_editor(
            display_df.head(40),
            column_config={"WhatsApp Link": st.column_config.LinkColumn(" 🎛️
Manual Dispatch", display_text="Open Chat Link")},
            disabled=["ID", "Name", "Blood Group", "Phone", "Days Rested"],
            use_container_width=True, hide_index=True
          )
```

```
            else:
                st.error("No active donor matches available for today's required scheduled blood configurations.")
        else:
            st.error("No active requirements or matching donors for today.")
            display_df = pd.DataFrame()

if current_day_name != "Sunday" and 'display_df' in locals() and not display_df.empty:
    st.sidebar.markdown("---")
    st.sidebar.subheader("🚀 Bulk Automation Pipeline")
    max_batch = min(len(display_df), 15)
    batch_size = st.sidebar.slider("Select Batch Size to Notify", 1, max_batch if max_batch > 1 else 2, 5)

    if st.sidebar.button(f"⚡ Launch Batch Trigger ({batch_size} Donors)"):
        progress_bar = st.sidebar.progress(0)
        status_text = st.sidebar.empty()
        batch_targets = display_df.head(batch_size)
        for index, (_, target) in enumerate(batch_targets.iterrows()):
            status_text.text(f"Opening WhatsApp terminal for: {target['Name']}...")
            webbrowser.open(target['WhatsApp Link'])
            time.sleep(1.5)
            progress_bar.progress(int((index + 1) / batch_size * 100))
        status_text.text("✅ Batch processing finished successfully!")

with tab2:
    st.subheader("📚 Global Institutional Registries")
    exp1, exp2 = st.columns(2)
```

```
with exp1:
    st.markdown("#### Complete Registered Transfusion Donors Pool")
    st.dataframe(df_donors, use_container_width=True, hide_index=True)
with exp2:
    st.markdown("#### Complete Registered Chronic Thalassemia Patients")
    st.dataframe(df_patients, use_container_width=True, hide_index=True)
```